Technical Report: Financial Report Analyzer

1. Executive Summary

The Financial Report Analyzer is a full-stack, Retrieval-Augmented Generation (RAG) application designed to allow users to upload, index, and semantically query lengthy financial documents (such as 10-K and 10-Q reports). The system is built with a focus on a professional, enterprise-grade aesthetic, ensuring zero "AI bloat" or flashy interfaces. It strictly prioritizes data accuracy, speed, and clear source citations.

2. Description of the Document/Data Source

The primary data sources for this application are financial and corporate documents (e.g., 10-K, 10-Q, Annual Reports). For testing, a synthetic "Acme Technologies Inc. - Q3 2026 Financial Results" document was used. It contains dense numerical data, segment performance breakdowns, and risk factors - ideal for testing RAG's ability to extract specific metrics without hallucination.

3. Architecture & Langchain Integration

The application leverages the following architecture and LangChain components:

- **VectorStoreRetriever:** FAISS (Facebook AI Similarity Search) running in-memory is used to index and retrieve the top K=3 most relevant chunks.
- **Chunking:** The document is split into smaller, overlapping chunks using LangChain's RecursiveCharacterTextSplitter (300 characters, 50 overlap) to preserve contextual boundaries around financial figures.
- **Embedding:** Each chunk is vectorized using the HuggingFace all-MiniLM-L6-v2 model.
- **PromptTemplate:** A strict prompt instructs the LLM to act as a financial analyst, avoid hallucinations, and explicitly state if the answer is missing.
- **LCEL / Chain Composition:** LangChain connects the retriever, context formatter, prompt, and LLM sequentially.
- **OutputParser:** A StructuredOutputParser enforces a JSON schema requiring two keys: analytical_answer and source_citations, ensuring UI separation of the generated text and the verbatim source.

- **LLM Provider:** Groq API (using openai/gpt-oss-20b) integrated via their OpenAI compatibility layer to provide fast, cost-effective inference.

4. Test Scenarios and Grounding Evaluation

The application was evaluated using 5 test queries against the Acme Technologies Q3 2026 report:

Test Question 1: What was the total consolidated revenue for Q3 2026?

- **Result:** $4.85 billion.
- **Evaluation:** Successfully grounded. The citation accurately points to the exact sentence in the executive summary.

Test Question 2: How much did the company spend on share repurchases?

- **Result:** $500 million for 4.2 million shares.
- **Evaluation:** Successfully grounded. Both the dollar amount and share count were correctly retrieved.

Test Question 3: What are the main risk factors mentioned?

- **Result:** Macroeconomic headwinds (strong USD), semiconductor supply chain constraints, and regulatory scrutiny (EU AI Act).
- **Evaluation:** Successfully grounded. Summarized effectively across multiple chunks.

Test Question 4: Who is the CEO of Acme Technologies?

- **Result:** 'I do not have enough information to answer this.'
- **Evaluation:** Successfully grounded (Negative test). The prompt successfully prevented hallucination when the context lacked the information.

Test Question 5 (Poor Performance Example): What is the gross margin percentage minus the R&D; expenses as a percentage of total revenue?

- **Result:** The LLM occasionally struggles to perform this multi-step mathematical reasoning correctly across disparate chunks. It may retrieve the correct individual numbers (64.9% and 24.7%) but fail to synthesize the final calculation reliably.

- **Evaluation:** Retrieval was successful, but Generation (reasoning) performed poorly on this complex math instruction.

5. Challenges & Resolutions

- **API Quotas & Model Deprecation:** The initial implementation attempted to use OpenAI, which failed due to billing requirements (429 Quota Exhausted).
- **Solution:** We migrated the LangChain ChatOpenAI wrapper to utilize Groq's OpenAI compatibility endpoint and dynamically fetch available open-source models, ensuring a free and robust solution.

6. Individual Contribution Statement

- **Siddhant Nikam (Data and Retrieval):** Configured the HuggingFace embeddings, FAISS vector store, and optimized the RecursiveCharacterTextSplitter chunking strategy.
- **Eshan Mohammed (Architecture):** Developed the FastAPI backend, integrated the Groq LLM API, designed the LangChain LCEL pipeline, and wired the React frontend endpoints.
- **Tvisha Majithia (Testing and Documentation):** Designed the minimal React UI, evaluated the RAG pipeline against the 5 test queries, wrote the video demo script, and compiled this final technical report.